Разработка программного обеспечения игры «Тетрис»

Проектирование программного обеспечения игры «Тетрис»

Программное обеспечение игры «Тетрис» (далее - программа) является распределённым, то есть работает на независимых физических устройствах, объединённых в единую систему.

Игровой процесс отображается на сенсорном экране Kinco, управление производится при помощи физических контроллеров, подключенных к ПЛК CP1L. В роли физических контроллеров выступают тумблеры. Часть программы, реализующая управление игрой при помощи тумблеров, программируется на языке LD. Графическая составляющая игры реализуется при помощи конструктора среды разработки «Kinco HMIware». Логика работы игры представлена макросом на языке Си.

Управление представлено 3 входящими сигналами, подаваемыми с тумблеров. Первый сигнал указывает программе на то, что падающую фигуру необходимо передвинуть на одну ячейку вправо. Второй сигнал по аналогии передвигает фигуру влево. Последний сигнал отвечает за поворот фигуры по часовой стрелке на 90 градусов. В целях упрощения реализации программы наличие управляющего сигнала для ускорения фигуры не предусматривается.

Для хранения состояния программы используется память панели Kinco, а для трансляции сигналов с тумблеров используется несколько ячеек памяти контроллера CP1L.

Помимо управления игрой также предусмотрена возможность сброса игры – то есть очистки игрового поля и обнуления счёта. Использование данной функции реализуется при помощи отдельной сенсорной кнопки на панели Kinco.

Санкт-Петербург
2024

На рисунке 14 показана структурная диаграмма программы.

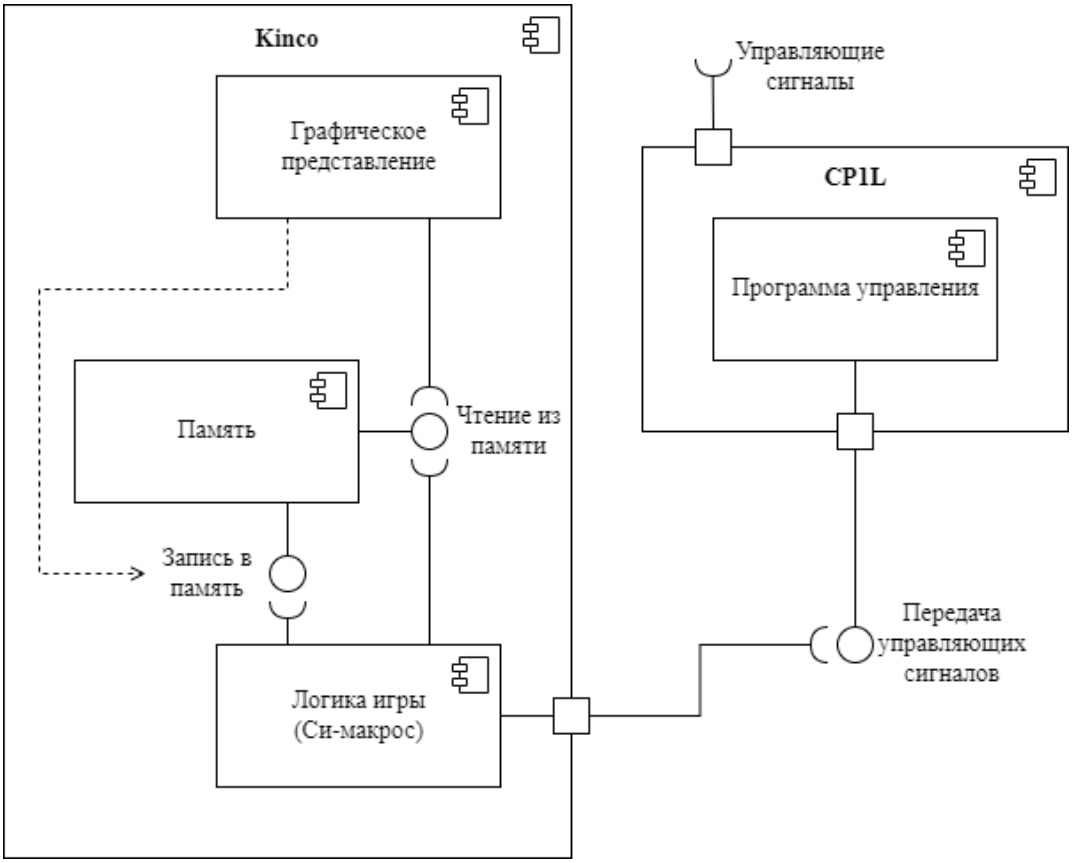

Рисунок 14 – Диаграмма компонентов программы

Реализация игры «Тетрис»

Программа для Kinco

Программа для панели Kinco состоит из графического интерфейса игры, макроса игровой логики и макроса сброса состояния. Графический интерфейс создаётся при помощи конструктора среды разработки Kinco HMIware. Макросы создаются на языке Си при помощи встроенного редактора.

Разработанный интерфейс представлен на рисунке 18. Интерфейс состоит из игрового поля 10 на 20 клеток (1), поля отображения следующей фигуры (2), полей отображения текущего счёта и индикации конца игры (3), кнопки сброса игры (4), невидимых элементов-таймеров (5), отвечающих на запуск макросов логики и сброса игры, а также кнопок, эмулирующих работу тумблеров (6).

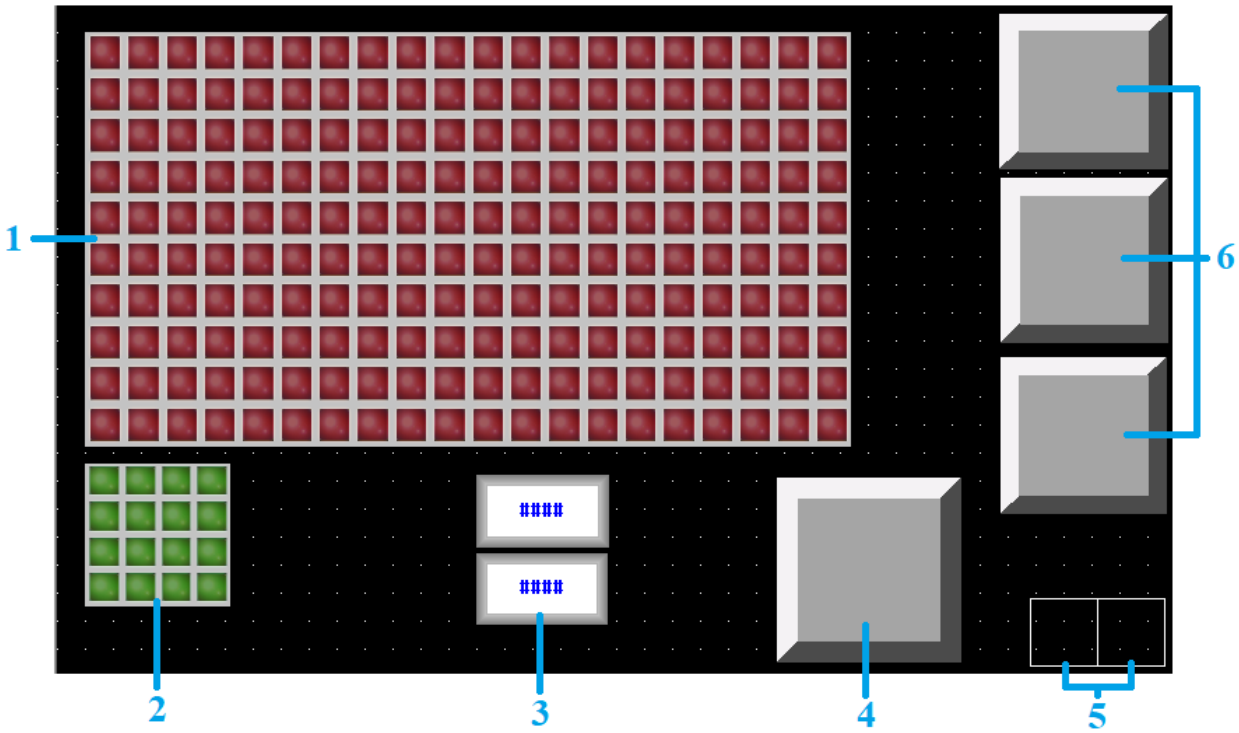

Рисунок 15 – Разработанный интерфейс игры

Игровое поле представлено 240 ячейками, 200 из которых отображаются на экране, а 40 являются скрытой областью, в которой генерируются фигуры. Состояние игрового поля расположено в участке памяти “LW” и занимает 240 байт, начиная с адреса 200.

Для отображения следующей фигуры используется ещё 16 байт, расположенных, начиная с адреса “LW” 692.

Игровая логика представлена си-макросом. Переменные макроса привязываются к определённым ячейкам памяти в специальной вкладке «Parameters», меню настройки памяти представлено на рисунке 16.

Parameters[macro_1.c]

DataType	Param name	PLC No.	PLC Addres...	Address	Word...	OptMode	Array	Array Length
unsigned short	view		LW	200	1	Read/Write	Yes	240
unsigned short	data		LW	440	1	Read/Write	Yes	240
signed short	figureX		LW	680	1	Read/Write	No	
signed short	figureY		LW	681	1	Read/Write	No	
unsigned short	figureType		LW	682	1	Read/Write	No	
unsigned short	figureRotation		LW	683	1	Read/Write	No	
unsigned short	timeCounter		LW	684	1	Read/Write	No	
unsigned short	right	0	H_word	50	1	Read/Write	No	
unsigned short	left	0	H_word	51	1	Read/Write	No	
unsigned short	rotate	0	H_word	52	1	Read/Write	No	
unsigned short	score		LW	688	1	Read/Write	No	
unsigned short	fail		LW	689	1	Read/Write	No	
unsigned short	nextFigure		LW	690	1	Read/Write	No	
unsigned short	nextRotation		LW	691	1	Read/Write	No	
unsigned short	next		LW	692	1	Read/Write	Yes	16

Рисунок 16 – Выделенная память

Для управления игрой используются 3 байта памяти контроллера CP1L в области памяти “H”, куда записывается состояние тумблеров.

Прочая информация, вроде текущего положения фигуры, игрового счёта, следующей фигуры и пр. расположена в участке памяти “LW” в диапазоне адресов 680-691.

Программный код макроса логики игры, а также дополнительного макроса, сбрасывающего состояние программы, представлены в приложении А.

Программа для CP1L

Часть программы, выполняемой на ПЛК CP1L, выполняет только одну функцию – транслирует сигналы с тумблеров в ячейки памяти, считываемые панелью Kinco. Программа представлена на рисунке 17.

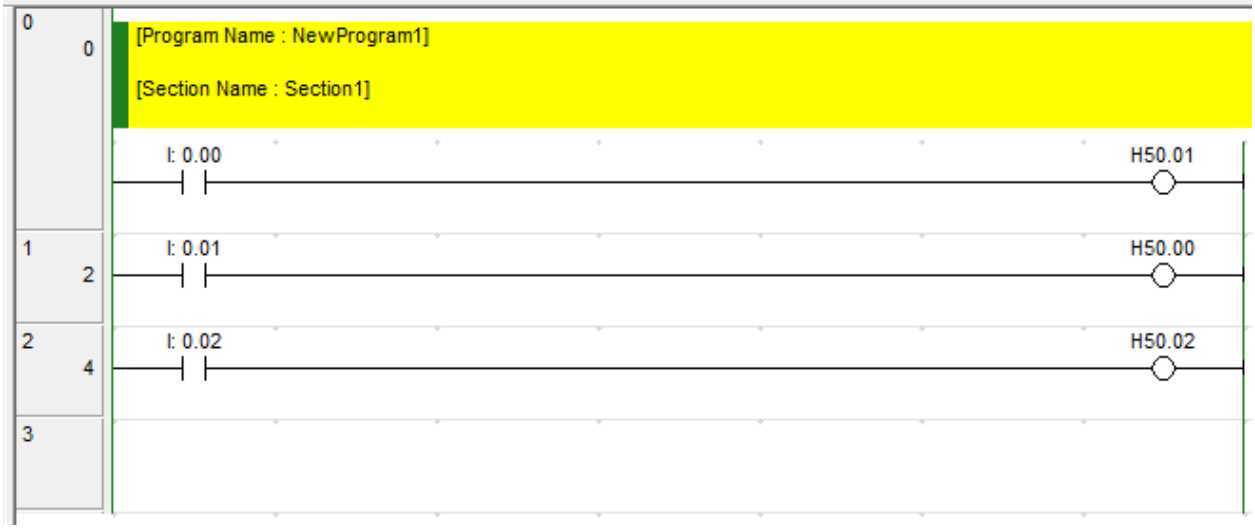

Рисунок 17 – Программа для контроллера CP1L

Вход 0.00 соответствует тумблеру, управляющему движением влево. 0.01 — движению вправо. 0.02 — вращению фигуры.

2.3 Демонстрация работы

На рисунках 18–20 показаны начало игры, состояние игры после получения нескольких очков и состояние игры после её завершения.

Изображения получены в результате симуляции работы физических устройств.

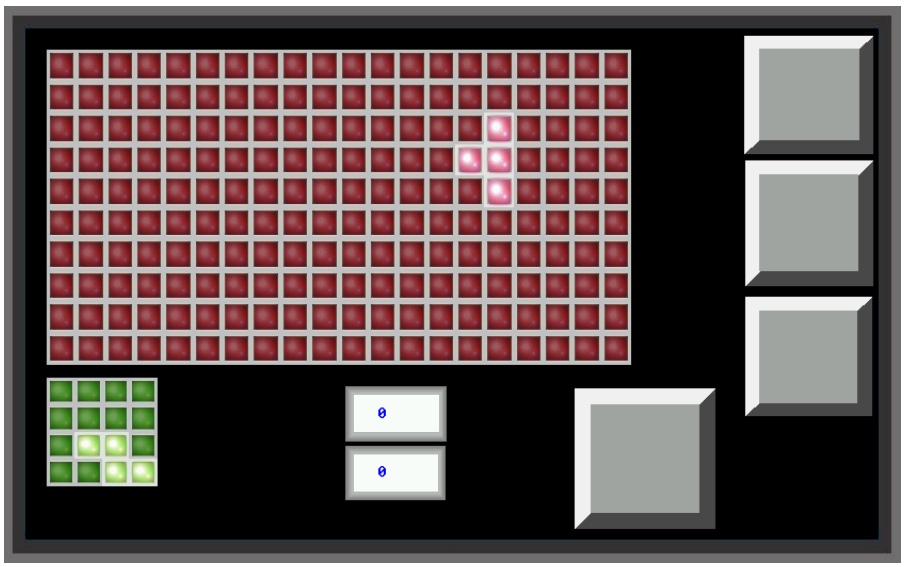

Рисунок 18 – Начало игры

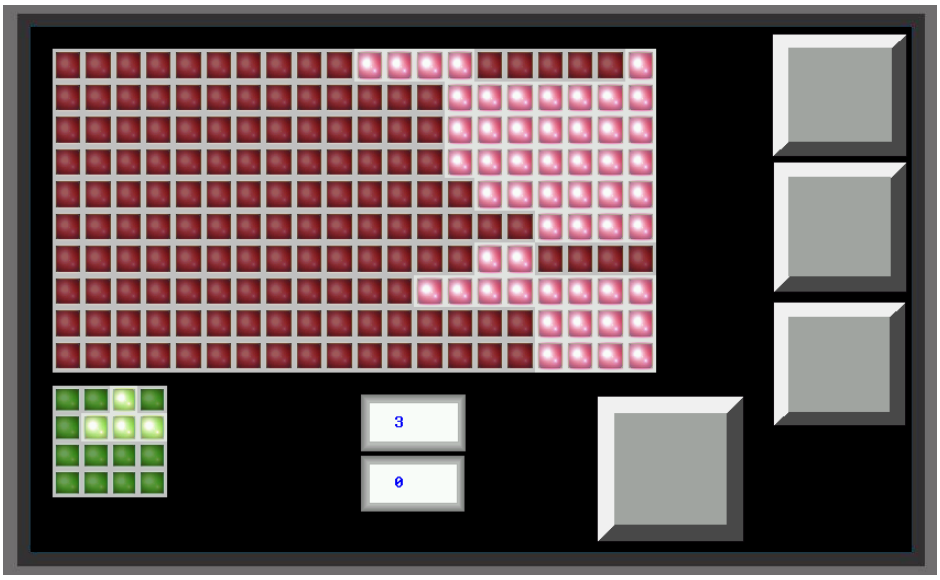

Рисунок 19 – Игрок получил 3 очка

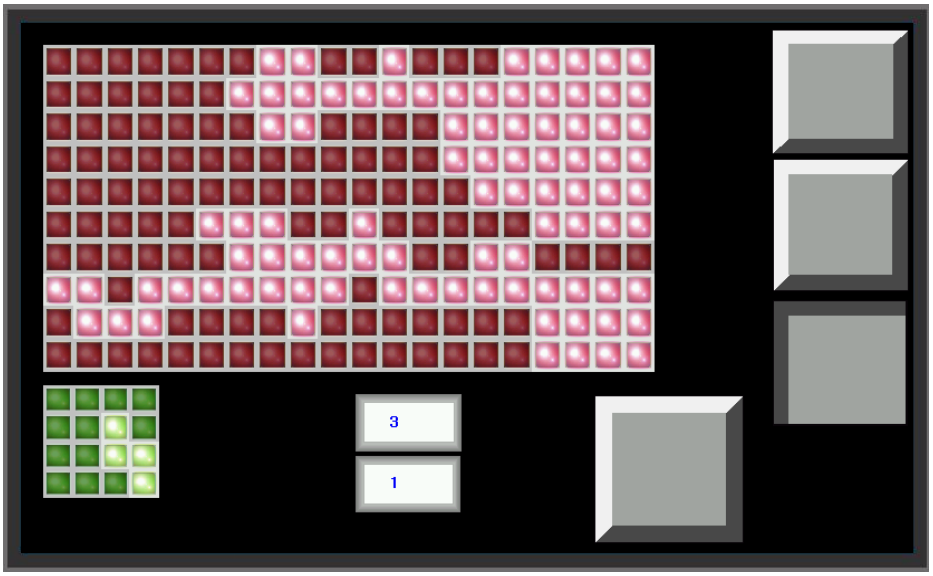

Рисунок 20 – Игра завершена

Приложение А

Программный код си-макросов

macro_1.c

```
#include "macrotypedef.h"
#include "math.h"
#include "stdlib.h"
#include "time.h"

#define WIDTH 10
#define HEIGHT 24

#define FIGURES_AMOUNT 7
#define FIGURES_ROTATIONS 4
#define FIGURE_SIZE 4

short figureIsActive(unsigned short* field)
{
	int index = 0;
	short isActive = 0;
	for (index = 0; index != WIDTH * HEIGHT; index++)
	{
		if (field[index] == 2)
		{
			isActive = 1;
		}
	}
	return isActive;
}

void generateFigure(unsigned short* x, unsigned short* y, unsigned short* type, unsigned short* rotation,
unsigned short* nFigure, unsigned short* nRotation)
{
	srand(time(NULL));
	*x = rand() % 6;
	*y = 0;
	*type = *nFigure;
	*rotation = *nRotation;
	srand(time(NULL));
	*nFigure = rand() % FIGURES_AMOUNT;
	srand(time(NULL));
	*nRotation = rand() % FIGURES_ROTATIONS;
}

void clearFigure(unsigned short* field)
{
	int index = 0;
	for (index = 0; index != WIDTH * HEIGHT; index++)
	{
		if (field[index] == 2)
		{
			field[index] = 0;
		}
	}
}

void placeFigure(unsigned short* field, unsigned short figures[][FIGURES_ROTATIONS][16], short* x, short*
y, unsigned short* type, unsigned short* rotation)
{
	unsigned short inFigureX;
	unsigned short inFigureY;
	unsigned short typeSize = FIGURES_ROTATIONS * FIGURE_SIZE * FIGURE_SIZE;
	unsigned short rotationSize =  FIGURE_SIZE * FIGURE_SIZE;
	clearFigure(field);
	for (inFigureX = 0; inFigureX != FIGURE_SIZE; inFigureX++)
	{
		for (inFigureY = 0; inFigureY != FIGURE_SIZE; inFigureY++)
		{
			if (((*x) + inFigureX >= 0) && ((*x) + inFigureX < WIDTH))
			{
				if (field[((*x) + inFigureX) * HEIGHT + ((*y) + inFigureY)] == 0)
```

```
                                {
                                        field[((*x)  +  inFigureX)  *  HEIGHT  +  ((*y)  +  inFigureY)]  =
figures[*type][*rotation][inFigureX * FIGURE_SIZE + inFigureY];
                                }
                        }
                }
        }
}

void show(unsigned short* field, unsigned short* display)
{
        unsigned short index = 0;
        for (index = 0; index != WIDTH * HEIGHT; index++)
        {
                display[index] = field[index] > 0 ? 1 : 0;
        }
}

short figureIsMovable(unsigned short* field)
{
        short isMovable = 1;
        int x;
        int y;
        for (x = 0; x != WIDTH; x++)
        {
                for (y = 0; y != HEIGHT; y++)
                {
                        if ((y != HEIGHT - 1) && (field[x * HEIGHT + y] == 2) && (field[x * HEIGHT + y +
1] == 1))
                        {
                                isMovable = 0;
                        }
                        else if ((y == HEIGHT - 1) && (field[x * HEIGHT + y] == 2))
                        {
                                isMovable = 0;
                        }
                }
        }
        return isMovable;
}

void moveFigure(unsigned short* y)
{
        *y = *y + 1;
}

void freezeFigure(unsigned short* field)
{
        int index = 0;
        for (index = 0; index != WIDTH * HEIGHT; index++)
        {
                if (field[index] == 2)
                {
                        field[index] = 1;
                }
        }
}

void shiftFigure(unsigned short* field,  short* xFigure, short direction)
{
        short isMovable = 1;
        short x;
        short y;
        for (x = 0; x != WIDTH; x++)
        {
                for (y = 0; y != HEIGHT; y++)
                {
                        if (field[x * HEIGHT + y] == 2)
                        {
                                if ((x + direction != -1) && (x + direction != WIDTH) && (field[(x +
direction) * HEIGHT + y] == 1))
                                {
                                        isMovable = 0;
                                }
                                else if ((x + direction == -1) || (x + direction == WIDTH))
                                {
```

```
                                        isMovable = 0;
                                }
                        }
                }
        }
        if (isMovable)
        {
                *xFigure = *xFigure + direction;
        }
}

void rotateFigure(unsigned short* field, unsigned short figures[][FIGURES_ROTATIONS][16], short* x, short*
y, unsigned short* type, unsigned short* rotation)
{
        *rotation = (*rotation + 1) % 4;
        short rotatable = 1;
        short inFigureX;
        short inFigureY;
        for (inFigureX = 0; inFigureX != FIGURE_SIZE; inFigureX++)
        {
                for (inFigureY = 0; inFigureY != FIGURE_SIZE; inFigureY++)
                {
                        if (figures[*type][*rotation][inFigureX * FIGURE_SIZE + inFigureY] == 2)
                        {
                                if ((*y) + inFigureY == HEIGHT)
                                {
                                        rotatable = 0;
                                }
                                else if ((*x) + inFigureX < 0 || (*x) + inFigureX >= WIDTH)
                                {
                                        rotatable = 0;
                                }
                                else if (field[((*x) + inFigureX) * HEIGHT + ((*y) + inFigureY)] == 1)
                                {
                                        rotatable = 0;
                                }
                        }
                }
        }

        if (rotatable == 0)
        {
                *rotation = (*rotation + 3) % 4;
        }
}

void checkRows(unsigned short* field, short* scoree)
{
        short x;
        short y;
        for (y = 0; y != HEIGHT; y++)
        {
                short full = 1;
                for (x = 0; x != WIDTH; x++)
                {
                        if (field[x * HEIGHT + y] != 1)
                        {
                                full = 0;
                        }
                }
                if (full)
                {
                        *scoree = *scoree + 1;
                        short yy;
                        short xx;
                        for (yy = y; yy != 0; yy--)
                        {
                                for (xx = 0; xx != WIDTH; xx++)
                                {
                                        field[xx * HEIGHT + yy] = field[xx * HEIGHT + yy - 1];
                                }
                        }
                }
        }
}
```

```
void checkFail(unsigned short* field, short* faiil)
{
        short x;
        short y;
        short isFail = 0;
        for (y = 0; y != 4; y++)
        {
                for (x = 0; x != WIDTH; x++)
                {
                        if (field[x * HEIGHT + y] == 1)
                        {
                                isFail = 1;
                        }
                }
        }

        *faiil = isFail;
}

int MacroEntry()
{
                if (fail)
                {
                        return 0;
                }
                unsigned short figures[FIGURES_AMOUNT][FIGURES_ROTATIONS][16] = {
                        {
                                {
                                        0, 0, 0, 0,
                                        0, 2, 2, 2,
                                        0, 0, 2, 0,
                                        0, 0, 0, 0
                                },
                                {
                                        0, 0, 2, 0,
                                        0, 2, 2, 0,
                                        0, 0, 2, 0,
                                        0, 0, 0, 0
                                },
                                {
                                        0, 0, 2, 0,
                                        0, 2, 2, 2,
                                        0, 0, 0, 0,
                                        0, 0, 0, 0
                                },
                                {
                                        0, 0, 2, 0,
                                        0, 0, 2, 2,
                                        0, 0, 2, 0,
                                        0, 0, 0, 0
                                }
                        },
                        {
                                {
                                        0, 0, 0, 0,
                                        0, 0, 2, 0,
                                        0, 2, 2, 0,
                                        0, 2, 0, 0
                                },
                                {
                                        0, 0, 0, 0,
                                        0, 0, 0, 0,
                                        0, 2, 2, 0,
                                        0, 0, 2, 2
                                },
                                {
                                        0, 0, 0, 0,
                                        0, 0, 2, 0,
                                        0, 2, 2, 0,
                                        0, 2, 0, 0
                                },
                                {
                                        0, 0, 0, 0,
                                        0, 0, 0, 0,
                                        0, 2, 2, 0,
                                        0, 0, 2, 2
```

```
		}
	},
	{
		{
			0, 0, 0, 0,
			0, 2, 2, 0,
			0, 0, 2, 0,
			0, 0, 2, 0
		},
		{
			0, 0, 0, 0,
			0, 0, 0, 2,
			0, 2, 2, 2,
			0, 0, 0, 0
		},
		{
			0, 0, 0, 0,
			0, 0, 2, 0,
			0, 0, 2, 0,
			0, 0, 2, 2
		},
		{
			0, 0, 0, 0,
			0, 0, 0, 0,
			0, 2, 2, 2,
			0, 2, 0, 0
		}
	},
	{
		{
			0, 0, 0, 0,
			0, 0, 2, 2,
			0, 0, 2, 0,
			0, 0, 2, 0
		},
		{
			0, 0, 0, 0,
			0, 0, 0, 0,
			0, 2, 2, 2,
			0, 0, 0, 2
		},
		{
			0, 0, 0, 0,
			0, 0, 2, 0,
			0, 0, 2, 0,
			0, 2, 2, 0
		},
		{
			0, 0, 0, 0,
			0, 2, 0, 0,
			0, 2, 2, 2,
			0, 0, 0, 0
		}
	},
	{
		{
			0, 0, 0, 0,
			0, 2, 2, 0,
			0, 2, 2, 0,
			0, 0, 0, 0
		},
		{
			0, 0, 0, 0,
			0, 2, 2, 0,
			0, 2, 2, 0,
			0, 0, 0, 0
		},
		{
			0, 0, 0, 0,
			0, 2, 2, 0,
			0, 2, 2, 0,
			0, 0, 0, 0
		},
		{
			0, 0, 0, 0,
			0, 2, 2, 0,
```

```
                                0, 2, 2, 0,
                                0, 0, 0, 0
                        }
                },
                {
                        {
                                0, 0, 0, 0,
                                2, 2, 2, 2,
                                0, 0, 0, 0,
                                0, 0, 0, 0
                        },
                        {
                                0, 0, 2, 0,
                                0, 0, 2, 0,
                                0, 0, 2, 0,
                                0, 0, 2, 0
                        },
                        {
                                0, 0, 0, 0,
                                2, 2, 2, 2,
                                0, 0, 0, 0,
                                0, 0, 0, 0
                        },
                        {
                                0, 0, 2, 0,
                                0, 0, 2, 0,
                                0, 0, 2, 0,
                                0, 0, 2, 0
                        }
                },
                {
                        {
                                0, 0, 0, 0,
                                0, 0, 2, 0,
                                0, 0, 2, 2,
                                0, 0, 0, 2
                        },
                        {
                                0, 0, 0, 0,
                                0, 0, 0, 0,
                                0, 0, 2, 2,
                                0, 2, 2, 0
                        },
                        {
                                0, 0, 0, 0,
                                0, 0, 2, 0,
                                0, 0, 2, 2,
                                0, 0, 0, 2
                        },
                        {
                                0, 0, 0, 0,
                                0, 0, 0, 0,
                                0, 0, 2, 2,
                                0, 2, 2, 0
                        }
                }
        };
        int x;
        int y;
        if (left)
        {
                if (figureIsActive(&(data[0])))
                {
                        shiftFigure(&(data[0]), &figureX, 1);
                        placeFigure(&(data[0]),   figures,   &figureX,   &figureY,   &figureType,
&figureRotation);
                }
        }

        if (right)
        {
                if (figureIsActive(&(data[0])))
                {
                        shiftFigure(&(data[0]), &figureX, -1);
                        placeFigure(&(data[0]),   figures,   &figureX,   &figureY,   &figureType,
&figureRotation);
```

```
                        }
                }

                if (rotate)
                {
                        if (figureIsActive(&(data[0])))
                        {
                                rotateFigure(&(data[0]),   figures,   &figureX,   &figureY,   &figureType,
&figureRotation);
                                placeFigure(&(data[0]),   figures,   &figureX,   &figureY,   &figureType,
&figureRotation);
                        }
                }

                if (timeCounter % 3 == 0)
                {
                        if (figureIsActive(&(data[0])))
                        {
                                if (figureIsMovable(&(data[0])))
                                {
                                        moveFigure(&figureY);
                                        placeFigure(&(data[0]), figures, &figureX, &figureY, &figureType,
&figureRotation);
                                }
                                else
                                {
                                        freezeFigure(&(data[0]));
                                        checkRows(&(data[0]), &score);
                                        checkFail(&(data[0]), &fail);
                                }
                        }
                        else
                        {
                                generateFigure(&figureX,    &figureY,    &figureType,    &figureRotation,
&nextFigure, &nextRotation);
                                placeFigure(&(data[0]),   figures,   &figureX,   &figureY,   &figureType,
&figureRotation);
                                short nX;
                                short nY;
                                for (nX = 0; nX != FIGURE_SIZE; nX++)
                                {
                                        for (nY = 0; nY != FIGURE_SIZE; nY++)
                                        {
                                                next[nX       *       FIGURE_SIZE       +       nY]       =
figures[nextFigure][nextRotation][nX * FIGURE_SIZE + nY] / 2;
                                        }
                                }
                        }
                }

                show(&(data[0]), &(view[0]));

                timeCounter++;
                if (timeCounter >= 10000)
                {
                        timeCounter = 0;
                }
                return 0;
}
```

init.c

```
#include "macrotypedef.h"
#include "math.h"

int MacroEntry()
{
        int index = 0;
        for (index = 0; index != 700; index++)
        {
                data[index] = 0;
        }
        return 0;
}
```